

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 03 | Arrays: Static Arrays, Dynamic Arrays and 2D Matrices

Week 3 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 03 at a glance

Item	Details
Lab title	Arrays: Static Arrays, Dynamic Arrays and 2D Matrices
Week	3 of 16
Prerequisites	Lab 01 (Python basics), Lab 02 (Big-O)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab03_Instructor.ipynb , Lab03_Student.ipynb
Next lab	Lab 04: Searching Techniques (Linear, Binary, Two Pointers, Sliding Window)

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain how an array is stored in **contiguous memory** and why index access is **$O(1)$** .
2. Calculate the memory address of any element using the **address formula**.
3. Perform and analyse the array operations: **traverse, access, update, search, insert, delete**.
4. Write a simple **Python class** and use it to build your own **Dynamic Array** with automatic resizing.
5. Explain **amortised $O(1)$** append and why Python lists double their capacity.
6. Work with **2D arrays (matrices)**: traversal, row/column sums, transpose and multiplication.
7. Apply arrays to real problems such as rotating, merging and cleaning data.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain the organisation of arrays in memory	Understand (C2)
CLO-2	Implement array operations and a dynamic array	Apply (C3)
CLO-3	Analyse the complexity of array operations	Analyse (C4)

2. Motivation: The Most Important Data Structure

If you learn only one data structure in your life, it should be the **array**. It is the foundation of almost everything else: strings, stacks, queues, heaps, hash tables, images, matrices and the tensors used in deep learning are all built on arrays.

2.1 Arrays in Daily Life

Real thing	How it is an array
A row of lockers numbered 0, 1, 2 ...	Each locker holds one item; you go directly to locker number 7
Seats in a cinema hall (Row C, Seat 12)	A 2D array: row and column
Mobile phone photo (e.g., 1080 × 1920)	A 2D array of pixels; with colour it is 3D (R, G, B)
Excel sheet / result gazette	A 2D table of rows (students) and columns (subjects)

Real thing	How it is an array
Daily temperature of Lahore for a month	A 1D array of 30 numbers
Chess / Ludo board	An 8×8 or 15×15 grid

2.2 Arrays in Industry

- **Image and video processing** (Instagram filters, Snapchat lenses) apply mathematics to huge 2D/3D arrays of pixels.
- **Databases** store table rows in pages that are essentially arrays on disk.
- **Games** use 2D arrays for maps and boards; **spreadsheets** are 2D arrays with formulas.
- Python's `list`, C++'s `std::vector` and Java's `ArrayList` are all **dynamic arrays**, used in nearly every program.

2.3 Arrays in Research

- **Deep Learning:** every image, sentence and model weight is stored as a multi-dimensional array (tensor) in NumPy, PyTorch and TensorFlow.
- **Remote Sensing & GIS:** satellite images (e.g., Sentinel-2, Landsat) are stacks of 2D arrays, one per spectral band; NDVI for crop health is computed element-by-element on arrays.
- **Scientific computing:** weather prediction and physics simulations operate on gigantic grids.

2.4 Why It Matters in 2026–2030

Arrays + hardware = speed

GPUs and AI chips are designed to process **arrays in parallel**. Because array elements sit next to each other in memory, the hardware can load them very quickly (cache-friendly). This is a big reason why array-based code is fast and why AI frameworks are built on arrays. Understanding arrays is your first step towards NumPy, Pandas, PyTorch and high-performance computing.

3. Part A: What Is an Array?

An **array** is a collection of elements of the **same type** stored in **contiguous (side-by-side) memory locations**. Each element is identified by an **index**, starting from **0**.

An array in memory

```

Index:      0      1      2      3      4
           +-----+-----+-----+-----+
marks:  | 67  | 89  | 45  | 92  | 78  |
           +-----+-----+-----+-----+
Address: 1000  1004  1008  1012  1016      (each int = 4 bytes)

```

3.1 Why Is Access $O(1)$? The Address Formula

Because elements are side by side and all have the same size, the computer can **calculate** the address of any element directly, without visiting the others:

**Address formula (1D)**

Address($A[i]$) = Base address + $i \times$ size of each element

Example: base = 1000, each int is 4 bytes. Address of $A[3] = 1000 + 3 \times 4 = 1012$. One multiplication and one addition, no matter how big the array is. That is **$O(1)$** .

This is also why indexes start at 0: index 0 means “0 elements after the base address”.

3.2 Static vs Dynamic Arrays

	Static array	Dynamic array
Size	Fixed when created	Grows and shrinks automatically
Examples	C/C++ <code>int a[10];</code>	Python <code>list</code> , C++ <code>vector</code> , Java <code>ArrayList</code>
If full	Cannot add more elements	Allocates a bigger block and copies elements
Memory	Exactly what you asked for	Keeps some extra capacity for future growth



4. Part B: Array Operations and Their Cost

We will now implement each operation **by hand**, as if we had only a fixed-size block of memory. This shows what Python lists do internally.

4.1 Traversal: Visit Every Element, $O(n)$

```
marks = [67, 89, 45, 92, 78]
for i in range(len(marks)):
    print(f"marks[{i}] = {marks[i]}")
```

4.2 Access and Update: $O(1)$

```
marks = [67, 89, 45, 92, 78]
print(marks[3])      # access:  $O(1)$ 
marks[3] = 95        # update:  $O(1)$ 
print(marks)
```

4.3 Insertion at a Position: $O(n)$

To insert at index **pos**, every element from **pos** to the end must **shift one place right** to make room. In the worst case (inserting at index 0) all n elements move.

Dry run

Insert 50 at index 1 in [67, 89, 45, 92, _]

```
Step 1: shift 92 right  -> [67, 89, 45, _ , 92]
Step 2: shift 45 right  -> [67, 89, _ , 45, 92]
Step 3: shift 89 right  -> [67, _ , 89, 45, 92]
Step 4: place 50        -> [67, 50, 89, 45, 92]
```

```
def insert_at(arr, size, pos, value):
    """arr has fixed capacity; size = number of used slots."""
    if size == len(arr):
        raise OverflowError("Array is full")
    for i in range(size, pos, -1): # shift right, starting from the end
        arr[i] = arr[i - 1]
    arr[pos] = value
    return size + 1

arr = [67, 89, 45, 92, None]          # capacity 5, size 4
size = insert_at(arr, 4, 1, 50)
print(arr, "size =", size)
```

4.4 Deletion at a Position: $O(n)$

To delete the element at index **pos**, every element after it **shifts one place left** to fill the gap.

```
def delete_at(arr, size, pos):
    for i in range(pos, size - 1):    # shift left
        arr[i] = arr[i + 1]
    arr[size - 1] = None              # clear the last used slot
    return size - 1

arr = [67, 50, 89, 45, 92]
size = delete_at(arr, 5, 1)
print(arr, "size =", size)
```

4.5 Linear Search: $O(n)$

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i                # found: return the index
    return -1                        # not found

print(linear_search([67, 89, 45, 92, 78], 92), linear_search([1, 2], 7))
```

Operation	Time	Why
Access / update a[i]	$O(1)$	Address formula
Traverse	$O(n)$	Visit every element
Search (unsorted)	$O(n)$	May check every element
Insert / delete at end	$O(1)$	No shifting needed
Insert / delete at beginning or middle	$O(n)$	Elements must shift



5. Part C: Python Classes in 5 Minutes

From now on we will build our own data structures. In Python, we do this with a **class**. A class is a **blueprint**; an object is a thing built from that blueprint.



Analogy

A class is like the **map of a house**. Using one map you can build many houses (objects). Each house has its own rooms (attributes) and things you can do in it (methods).

```

class Student:
    def __init__(self, name, roll):      # constructor: runs when an object is
created
        self.name = name                # attribute
        self.roll = roll
        self.marks = []

    def add_mark(self, m):               # method
        self.marks.append(m)

    def average(self):
        return sum(self.marks) / len(self.marks) if self.marks else 0

    def __str__(self):                  # how the object looks when printed
        return f"{self.roll}: {self.name} (avg {self.average():.1f})"

s = Student("Ayesha", "BSCS-21")
s.add_mark(80); s.add_mark(90)
print(s)

```

self refers to the object itself. **__init__** sets up the object, and **__str__** decides how it prints. We will use special methods like **__len__** and **__getitem__** so our structures behave like Python lists.



6. Part D: Build Your Own Dynamic Array

How does a Python list grow without limit if memory blocks are fixed? Answer: when the block is full, it **allocates a new block (usually about double the size), copies all elements, and frees the old block**.

Resizing by doubling

```

Capacity 2: [10, 20]          append 30 -> FULL!
New capacity 4: copy 10, 20  -> [10, 20, 30, _]
append 40                      -> [10, 20, 30, 40]
New capacity 8: copy 4 items -> [10, 20, 30, 40, 50, _, _, _]

```

```

class DynamicArray:
    def __init__(self):
        self.capacity = 1
        self.size = 0
        self.data = [None] * self.capacity    # our fixed block of memory

    def __len__(self):
        return self.size

    def __getitem__(self, i):
        if not 0 <= i < self.size:
            raise IndexError("index out of range")
        return self.data[i]

    def _resize(self, new_capacity):
        new_data = [None] * new_capacity
        for i in range(self.size):            # copy: O(n)
            new_data[i] = self.data[i]
        self.data = new_data
        self.capacity = new_capacity

    def append(self, value):
        if self.size == self.capacity:
            self._resize(2 * self.capacity)    # double when full
        self.data[self.size] = value
        self.size += 1

    def __str__(self):
        return str([self.data[i] for i in range(self.size)]) + f" (capacity {self.capacity})"

arr = DynamicArray()
for x in [10, 20, 30, 40, 50]:
    arr.append(x)
    print(arr)

```

6.1 Amortised $O(1)$: Why Doubling Is Clever

A resize costs $O(n)$, but it happens rarely. If we append n items, the total copying work is $1 + 2 + 4 + \dots + n/2 + n < 2n$. Spread over n appends, that is less than 2 **copies per append on average**, so each append is **amortised $O(1)$** .

```

class CountingArray(DynamicArray):
    copies = 0
    def _resize(self, new_capacity):
        CountingArray.copies += self.size
        super()._resize(new_capacity)

for n in [1_000, 10_000, 100_000]:
    CountingArray.copies = 0
    a = CountingArray()
    for i in range(n):
        a.append(i)
    print(f"n = {n:>7}, total copies = {CountingArray.copies:>7}  ({CountingArray.copies / n:.2f} per append)")

```

 What if we grew by +1 instead of $\times 2$?

Then every append would trigger a copy, and appending n items would cost $1 + 2 + \dots + n = \mathbf{O(n^2)}$. Doubling turns that into $O(n)$ total. A small design decision with a huge impact.

7. Part E: 2D Arrays (Matrices)

A **2D array** stores data in **rows and columns**. In Python we represent it as a **list of lists**.

```
#           Maths  Physics  CS
result = [ [ 78,      85,      90 ], # row 0: Ali
            [ 66,      72,      95 ], # row 1: Sara
            [ 88,      91,      79 ] ] # row 2: Hamza

print(result[1][2])                # Sara's CS marks -> 95
rows, cols = len(result), len(result[0])

for r in range(rows):               # row-wise traversal
    print("Row", r, "total =", sum(result[r]))

for c in range(cols):               # column-wise traversal
    col_total = 0
    for r in range(rows):
        col_total += result[r][c]
    print("Subject", c, "average =", round(col_total / rows, 1))
```

The classic 2D trap

`grid = [[0] * 3] * 3` creates **three references to the same row**. Changing `grid[0][0]` changes all rows! Always use `grid = [[0] * 3 for _ in range(3)]`.

```
wrong = [[0] * 3] * 3
wrong[0][0] = 9
print("wrong:", wrong)

right = [[0] * 3 for _ in range(3)]
right[0][0] = 9
print("right:", right)
```

7.1 Row-Major Storage and the 2D Address Formula

Memory is one-dimensional, so a 2D array is stored **row after row** (row-major order, used by C, C++ and NumPy by default).

Address formula (2D, row-major)

Address($A[i][j]$) = Base + ($i \times \text{number_of_columns} + j$) $\times$ element_size

Example: a 3×4 int matrix at base 2000. Address of $A[2][1] = 2000 + (2 \times 4 + 1) \times 4 = 2036$.

7.2 Transpose and Matrix Multiplication

```
def transpose(m):
    rows, cols = len(m), len(m[0])
    t = [[0] * rows for _ in range(cols)]
    for i in range(rows):
        for j in range(cols):
            t[j][i] = m[i][j]
    return t

def mat_mult(a, b):          # (n x m) * (m x p) -> (n x p):
    0(n*m*p)
    n, m, p = len(a), len(b), len(b[0])
    c = [[0] * p for _ in range(n)]
    for i in range(n):
        for j in range(p):
            for k in range(m):
                c[i][j] += a[i][k] * b[k][j]
    return c

A = [[1, 2, 3], [4, 5, 6]]
print(transpose(A))
print(mat_mult([[1, 2], [3, 4]], [[5, 6], [7, 8]]))
```

Matrix multiplication is the core operation of neural networks. A single layer computes **output = input × weights**, which is exactly this triple loop (done on GPUs in parallel).

8. C++ Corner

C++

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int marks[5] = {67, 89, 45, 92, 78};           // static array: fixed
    size                                           size
    cout << "Address of marks[0]: " << &marks[0] << endl;
    cout << "Address of marks[1]: " << &marks[1] << endl;    // 4 bytes
    later

    vector<int> v;                                // dynamic array
    for (int i = 0; i < 10; i++) {
        v.push_back(i);
        cout << "size=" << v.size() << " capacity=" << v.capacity() <<
    endl;
    }

    int grid[3][4] = {0};                         // 2D static array
    grid[2][1] = 7;
    cout << grid[2][1] << endl;
    return 0;
}
```

Run this and watch `capacity()` jump (1, 2, 4, 8, 16 on most compilers). That is the same doubling strategy you just implemented.

9. Common Mistakes

Mistake	Result	Fix
Accessing <code>a[len(a)]</code>	IndexError	Last valid index is <code>len(a) - 1</code>
Modifying a list while looping over it	Skipped elements	Loop over a copy or build a new list
<code>[[0]*c]*r</code> for a grid	All rows change together	<code>[[0]*c for _ in range(r)]</code>
Using <code>insert(0, x)</code> in a big loop	Hidden $O(n^2)$	Append, then reverse once
Mixing up rows and columns	Wrong results / IndexError	<code>m[row][col]</code> ; rows = <code>len(m)</code> , cols = <code>len(m[0])</code>

10. Lab Tasks

Solve all tasks in `Lab03_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Array Statistics in One Pass

Write `stats(arr)` that returns (minimum, maximum, average) using a **single loop** and no built-in `min`, `max` or `sum`. Return `None` for an empty array.

```
stats([67, 89, 45, 92, 78]) → (45, 92, 74.2)
```

Task 2 Move Zeroes to the End

Move all `0`s to the end of the array **in place** while keeping the order of the non-zero elements. Use $O(1)$ extra space. Return the same list.

```
[0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]
```

Task 3 Rotate an Array

Rotate the array to the right by `k` steps **in place** (`k` may be larger than `n`).

Hint (reversal trick): reverse the whole array, then reverse the first `k` elements, then reverse the rest. This is $O(n)$ time and $O(1)$ space.

```
rotate([1, 2, 3, 4, 5, 6, 7], 3) → [5, 6, 7, 1, 2, 3, 4]
```

Task 4 Merge Two Sorted Arrays

Two sections' merit lists are each sorted in ascending order. Write `merge_sorted(a, b)` that returns one sorted list in $O(n + m)$ time. Do not use `sort()` or `sorted()`. (This idea is the heart of Merge Sort in Lab 07.)

```
merge_sorted([1, 4, 9], [2, 3, 10, 12]) → [1, 2, 3, 4, 9, 10, 12]
```

Task 5 Complete the Dynamic Array

Extend the `DynamicArray` class from Part D (redefined in the notebook) with these methods:

`insert(pos, value)` (shift right, resize if full), `remove_at(pos)` (shift left and return the removed value), and `index_of(value)` (return the index or `-1`). Raise `IndexError` for invalid positions.

Task 6 Rotate an Image by 90°

A grayscale image is an $n \times n$ matrix. Rotate it **90° clockwise in place**.

Hint: transpose the matrix in place (swap `m[i][j]` with `m[j][i]` for $j > i$), then reverse each row.

```
[[1, 2, 3],      [[7, 4, 1],
 [4, 5, 6],    →  [8, 5, 2],
 [7, 8, 9]]      [9, 6, 3]]
```

Task 7 Spiral Order of a Matrix

Return all elements of an $m \times n$ matrix in **spiral order** (clockwise from the top-left). Keep four boundaries: top, bottom, left, right, and shrink them after each side.

```
[[1, 2, 3],
 [4, 5, 6],    → [1, 2, 3, 6, 9, 8, 7, 4, 5]
 [7, 8, 9]]
```



11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab03` folder.

#	Problem	Level	Concept practised
283	Move Zeroes	Easy	In-place two indexes (Task 2)
26	Remove Duplicates from Sorted Array	Easy	In-place overwrite
88	Merge Sorted Array	Easy	Merging (Task 4)
1572	Matrix Diagonal Sum	Easy	2D indexing
189	Rotate Array	Medium	Reversal trick (Task 3)
48	Rotate Image	Medium	Transpose + reverse (Task 6)
54	Spiral Matrix	Medium	Boundaries (Task 7)



12. Home Task

1. Finish all tasks and push `Lab03_Student.ipynb` to GitHub.
2. Add a `shrink` rule to `MyArray`: when size falls to one quarter of capacity, halve the capacity. Explain why we shrink at 1/4 and not at 1/2.
3. Write `matrix_add`, `is_symmetric` and `row_with_max_sum` functions for 2D arrays with tests.
4. **Mini-project: Result Sheet Analyser.** Store a class result (at least 10 students $\times$ 5 subjects) as a 2D array. Print each student's total, percentage and grade, each subject's average and topper, and the class position list.
5. Solve at least 3 LeetCode problems from the list.



13. Viva Questions

1. What is an array? Why must all elements be of the same type?
2. Why is accessing `a[i]` $O(1)$? Write the address formula.
3. Why is inserting at the beginning of an array $O(n)$?
4. Differentiate between static and dynamic arrays.
5. How does a dynamic array grow? Why double instead of adding 1?
6. What does amortised $O(1)$ mean?
7. What is row-major order? Calculate the address of `A[1][2]` in a 3×3 int array at base 500.
8. Why is `[[0]*3]*3` dangerous?
9. What is the time complexity of multiplying two $n \times n$ matrices with three loops?
10. Give two real applications of 2D arrays.



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.

15. References and Further Learning

- NumPy documentation: numpy.org/doc
- Python list implementation notes: docs.python.org/3/faq/design.html (How are lists implemented?)
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.